EMSI ÉCOLE MAROCAINE DES SCIENCES DE L'INGÉNIEUR

FORMATION D'INGÉNIEURS - INGÉNIERIE ET RÉSEAUX

PROJET DE FIN D'ANNÉE

3ème Année en Ingénierie Informatique et Réseaux

MANUGOM STOCK

Application Web de Gestion de Stock et de Commandes de Pneus

Rapport académique de conception, de réalisation et de validation

Réalisé par : À compléter

Filière : IIR - Ingénierie Informatique et Réseaux

Encadrant(e) : À compléter

Date de remise / soutenance : À compléter

EMSI - École Marocaine des Sciences de l'Ingénieur

Année universitaire : 2025/2026

Fiche d'identification

Champ	Valeur
Intitulé	MANUGOM Stock - gestion de stock et commandes de pneus
Nature	Projet de Fin d'Année - application web full-stack
Établissement	EMSI - École Marocaine des Sciences de l'Ingénieur
Filière	IIR - Ingénierie Informatique et Réseaux, 3ème année
Entreprise étudiée	MANUGOM SARL, Casablanca
Étudiant(e)	À compléter
Encadrant(e)	À compléter
Année universitaire	2025/2026
Technologies	Python, Django, SQLite, HTML, CSS, JavaScript, Bootstrap, Chart.js

Ce document décrit uniquement les fonctionnalités réellement présentes dans le projet. Les références, prix, comptes, clients et fournisseurs chargés par la commande seed_demo sont identifiés comme données de démonstration.

Remerciements

Ce projet a été réalisé dans un cadre académique de troisième année à l'EMSI.

Reconnaissance académique

Je tiens à remercier l'ensemble du corps enseignant de l'EMSI pour les connaissances transmises en ingénierie logicielle, développement web, bases de données, réseaux et sécurité. Ces enseignements ont fourni les méthodes nécessaires pour analyser un besoin, structurer une architecture et vérifier une solution par des tests reproductibles.

Accompagnement

Mes remerciements s'adressent également à l'encadrant(e) du projet, dont le nom sera ajouté avant la remise, pour ses orientations, ses retours et son suivi. Les remarques formulées pendant la réalisation ont permis de mieux distinguer les besoins métier, les contraintes techniques et les exigences de présentation académique.

Environnement professionnel

Je remercie enfin les personnes ayant facilité la compréhension du contexte de MANUGOM SARL et de ses activités liées aux pneumatiques. La réalisation de cette application a constitué une occasion concrète de consolider une démarche de conception, de développement et de validation complète.

Résumé

MANUGOM Stock est une application web académique dédiée à la gestion des pneus, des stocks et des commandes clients.

Contexte et solution

L'activité décrite nécessite de centraliser les références, les dimensions, les marques, les clients, les fournisseurs et les opérations qui font évoluer les quantités. La solution développée repose sur Django et sur une base SQLite unique. Elle distingue trois rôles : administrateur, employé et client. Le visiteur consulte le catalogue tandis que le client authentifié peut constituer un panier, enregistrer une commande et suivre son statut.

Fiabilité métier

Les opérations critiques sont encapsulées dans des services. La confirmation d'une commande vérifie une seconde fois le stock, retire les quantités dans une transaction et crée des mouvements traçables. L'annulation restaure le stock une seule fois lorsque les règles le permettent. La réception d'un approvisionnement augmente les quantités sans permettre de double réception.

Résultat

Le projet comprend une interface responsive jaune et noire, un tableau de bord alimenté par SQLite, cinq diagrammes UML, un MCD, un MLD, une documentation d'installation et une suite de 34 tests réussis. Le résultat constitue un prototype fonctionnel et démontrable ; les données générées ne sont pas des informations commerciales officielles.

- Mots-clés : Django, SQLite, pneus, stock, commande, approvisionnement, UML, tests.

Abstract

MANUGOM Stock is an academic web application for tyre inventory and customer-order management.

Context and solution

The application centralises tyre references, dimensions, brands, categories, customers, suppliers and stock operations. It is implemented with Django's MVT architecture and a single SQLite database. Three roles are supported: administrator, employee and customer. Public visitors can browse the catalogue, while authenticated customers can maintain a cart, submit an order and follow its status.

Business reliability

Critical updates are handled by dedicated services. Order confirmation performs a second stock check, updates quantities inside a short transaction and records traceable stock movements. Cancellation restores stock at most once when the transition is allowed. Supplier-order reception increases quantities and prevents duplicate reception.

Outcome

The deliverable includes a responsive automotive interface, a database-driven dashboard, five UML diagrams, conceptual and logical data models, installation documentation and 34 successful automated tests. The result is a functional demonstration prototype; generated records and prices are not official commercial information.

- Keywords: Django, SQLite, tyres, inventory, orders, procurement, UML, testing.

Liste des abréviations

Les sigles utilisés dans le rapport sont définis ci-dessous.

Abréviation	Signification
CRUD	Create, Read, Update, Delete
CSRF	Cross-Site Request Forgery
DBMS / SGBD	Système de gestion de base de données
EMSI	École Marocaine des Sciences de l'Ingénieur
FK	Foreign Key - clé étrangère
HTTP / HTTPS	Protocoles de communication Web
IIR	Ingénierie Informatique et Réseaux
MCD	Modèle conceptuel de données
MLD	Modèle logique de données
MVT	Model - View - Template
ORM	Object-Relational Mapping
PK	Primary Key - clé primaire
UML	Unified Modeling Language

Liste des figures

Les figures principales sont générées en vectoriel afin de rester nettes à l'impression.

N°	Figure	Page
1	Architecture Django MVT de MANUGOM Stock	24
2	Diagramme de cas d'utilisation global	25
3	Diagramme de classes	26
4	Diagramme de séquence de commande	27
5	Diagramme d'activité de commande	28
6	Diagramme de déploiement	29
7	MCD / vue relationnelle synthétique	30
8	Éléments d'identité visuelle	41
9	Répartition des tests	44

Liste des tableaux

Les tableaux synthétisent les choix, règles et résultats sans remplacer les explications.

N°	Tableau	Page
1	Fiche d'identification du projet	2
2	Liste des abréviations	6
3	Acteurs et responsabilités	18
4	Besoins fonctionnels	19
5	Exigences non fonctionnelles	20
6	Modèle logique simplifié	31
7	Extrait du dictionnaire de données	32
8	Technologies retenues	34
9	Résultats réels des tests	44
10	Comptes de démonstration	50

Sommaire

Introduction générale 11

Chapitre 1 - Contexte général 12

Présentation de l'EMSI et de MANUGOM SARL 13

Contexte et situation initiale 14

Problématique et objectifs 15

Méthodologie et périmètre 16

Chapitre 2 - Présentation et analyse du projet 17

Acteurs et responsabilités 18

Besoins fonctionnels 19

Besoins non fonctionnels 20

Règles de gestion 21

Critères d'acceptation 22

Chapitre 3 - Analyse et conception 23

Architecture MVT 24

Diagrammes UML 25

Sommaire - suite

MCD et modèle logique 30

Contraintes et dictionnaire 32

Chapitre 4 - Réalisation et développement 33

Environnement et technologies 34

Structure du projet 35

Authentification et sécurité 36

Catalogue et gestion des pneus 37

Panier et commandes 38

Stock et approvisionnements 39

Tableau de bord, devis et contact 40

Identité visuelle 41

Chapitre 5 - Tests et résultats 42

Tests, résultats et corrections 43

Guide utilisateur 47

Conclusion générale 48

Bibliographie et webographie 49

Annexes 50

Introduction générale

La gestion d'un stock de pneumatiques associe des contraintes commerciales, logistiques et techniques.

Contexte

Une référence de pneu ne se résume pas à un nom de produit. Elle associe une marque, une gamme, une largeur, une hauteur, un diamètre, un indice de charge, un indice de vitesse, une saison et un usage. Une erreur de référence ou de quantité peut perturber la préparation d'une commande et rendre l'historique difficile à interpréter.

Objectif du projet

Le projet MANUGOM Stock vise à proposer une application web centralisée permettant de consulter le catalogue, gérer les référentiels, traiter les commandes clients, recevoir des approvisionnements et tracer chaque mouvement de stock. Le système doit rester compréhensible pour une soutenance de troisième année tout en appliquant des principes professionnels.

Organisation du rapport

Le premier chapitre présente le contexte. Le deuxième formalise les acteurs et les besoins. Le troisième décrit l'architecture, l'UML et la base de données. Le quatrième expose la réalisation Django. Le cinquième présente les tests, les résultats et les limites. Des annexes regroupent les commandes de lancement, comptes et transitions principales.

01

Contexte général

Ce chapitre situe le projet dans son environnement académique et métier. Il présente l'EMSI, l'activité décrite de MANUGOM SARL, la situation initiale, la problématique et la méthode retenue pour conduire le développement.

Présentation de l'EMSI et de MANUGOM SARL

Le projet est réalisé dans le cadre de la troisième année de la filière Ingénierie Informatique et Réseaux.

Cadre académique

La formation mobilise des compétences transversales : programmation, bases de données, développement web, modélisation, systèmes, réseaux et sécurité. Un projet de fin d'année doit démontrer la capacité à partir d'un besoin, concevoir une solution cohérente, la réaliser, la tester et la documenter de façon reproductible.

Entreprise étudiée

MANUGOM SARL est présentée dans le cahier des charges comme une entreprise d'importation, de distribution et de vente de pneus et de jantes. Les coordonnées fournies sont le 222 Boulevard Dakhla à Casablanca, le 06 62 12 34 97 et l'adresse manugom.2000@gmail.com. Les horaires indiqués vont du lundi au samedi, de 08:30 à 18:30.

Précaution documentaire

Les marques mentionnées servent à préparer une démonstration de catalogue. Les références, prix, stocks et opérations créés par le script de données ne sont pas présentés comme une offre commerciale officielle. Cette distinction apparaît dans l'interface et dans le README.

Contexte et situation initiale

Le besoin naît de la diversité des références et du caractère critique des quantités disponibles.

Complexité du catalogue

Les dimensions telles que 205/55 R16 91V contiennent plusieurs informations distinctes. Une application doit conserver la dimension complète tout en enregistrant séparément largeur, hauteur, diamètre, charge et vitesse. Cette décomposition permet de filtrer précisément et de valider la cohérence des données.

Traçabilité

Une modification directe du stock ne suffit pas. Pour expliquer un écart, chaque entrée, sortie, retour ou ajustement doit conserver la quantité, le stock avant, le stock après, le motif, l'utilisateur et, si nécessaire, la commande ou l'approvisionnement associé.

Multiplicité des utilisateurs

Le visiteur recherche un produit, le client gère son panier et ses commandes, l'employé traite les opérations courantes et l'administrateur gère les utilisateurs et référentiels critiques. La séparation des rôles réduit les erreurs et protège les informations.

Problématique et objectifs

Comment centraliser la gestion des pneus et des commandes tout en garantissant cohérence, sécurité et traçabilité ?

Problématique

La solution doit empêcher le stock négatif, les doubles confirmations et les doubles réceptions. Elle doit conserver les prix historiques, protéger les commandes de chaque client et fournir une vision synthétique de l'activité sans dupliquer la logique sensible dans les vues.

Objectif général

Développer une application web Django connectée à une seule base SQLite, utilisable depuis un ordinateur, une tablette ou un téléphone, et couvrant le catalogue, le panier, les commandes, les clients, fournisseurs, approvisionnements, mouvements, alertes, devis et statistiques.

Objectifs mesurables

Le projet est considéré fonctionnel lorsque les pages principales rendent sans erreur, que les rôles sont contrôlés, que les transactions critiques produisent les mouvements attendus, que les données de démonstration sont cohérentes et que les tests automatisés passent réellement.

Méthodologie et périmètre

La démarche associe analyse, conception, réalisation incrémentale et validation continue.

Démarche

Le travail a commencé par l'analyse du cahier des charges et des références visuelles. Les entités et règles ont ensuite été traduites en modèles Django. Après les migrations, les services métier ont été développés avant les interfaces. Les données de démonstration et les tests ont permis de stabiliser le code avant la production de l'UML et du rapport.

Périmètre inclus

Le périmètre couvre l'authentification, les trois rôles, les CRUD demandés, le catalogue, le panier, les commandes, le stock, les approvisionnements, le tableau de bord, les alertes, les devis, le contact et l'administration Django.

Périmètre exclu

Aucune passerelle de paiement, API WhatsApp, API cartographique ou intégration de transporteur n'est configurée. Les boutons Appeler et WhatsApp utilisent des liens standards tel: et wa.me. Les fonctions d'export de données et la zone de carte ont été retirées à la demande du propriétaire du projet.

02

Présentation et analyse du projet

Ce chapitre transforme le besoin général en responsabilités, fonctions, contraintes et règles vérifiables. Il précise ce que chaque acteur peut faire et les conditions qui déterminent la validité d'une opération.

Acteurs et responsabilités

Quatre acteurs structurent le diagramme de cas d'utilisation.

Principe de moindre privilège

Chaque page de gestion est protégée par un décorateur. Les vues client utilisent la fiche Client liée à l'utilisateur connecté afin de filtrer les commandes et lignes de panier. Un compte employé ne dispose pas des fonctions réservées à l'administrateur.

Redirection selon le rôle

Après authentification, un gestionnaire est dirigé vers le tableau de bord alors qu'un client revient vers le catalogue. Le parcours d'ajout avant connexion renvoie désormais vers une page GET valide, ce qui évite l'erreur HTTP 405 observée puis corrigée.

Acteur	Responsabilités principales	Restrictions
Visiteur	Accueil, catalogue, recherche, marques, devis, contact	Pas de panier ni commande
Client	Profil, panier, validation, suivi et annulation autorisée	Accès à ses propres commandes seulement
Employé	Pneus, clients, commandes, mouvements, approvisionnements, alertes	Pas de gestion des administrateurs
Administrateur	Capacités employé + comptes, employés et référentiels	Actions critiques toujours contrôlées

Besoins fonctionnels

Les fonctions sont regroupées par domaine afin de maintenir une architecture lisible.

Catalogue et vente

Le client recherche une référence par marque, modèle, dimension, saison, prix ou disponibilité. Le total du panier est recalculé côté serveur. Au moment de la validation, les lignes de commande copient la référence, la marque, le modèle, la dimension et le prix unitaire pour préserver l'historique.

Gestion et pilotage

L'espace de gestion présente des tableaux opérationnels, des formulaires validés, des badges de statut et des alertes. Les statistiques et graphiques proviennent de requêtes sur SQLite ; aucune valeur n'est codée en dur pour simuler les résultats.

Module	Fonctions réalisées
Authentification	Inscription, connexion, déconnexion, profil, mot de passe, rôles
Catalogue	Recherche, filtres, tri, pagination, détail, disponibilité
Référentiels	Pneus, marques, catégories, clients, fournisseurs
Vente	Panier, quantités, validation, commande et transitions
Stock	Entrée, sortie, retour, ajustements et alertes
Achat	Approvisionnements, lignes, réception et total
Pilotage	Indicateurs, cinq graphiques, commandes récentes
Relation	Demandes de devis et messages de contact

Besoins non fonctionnels

La qualité d'une application de gestion dépend autant des règles que des écrans.

Fiabilité

Les écritures qui modifient plusieurs entités utilisent des transactions courtes. Cette précaution est particulièrement importante avec SQLite, qui convient à la démonstration mais supporte moins bien une forte concurrence d'écritures.

Clarté académique

Les noms sont explicites, les statuts sont définis par des choix Django, les scénarios critiques sont testés et les limites sont déclarées. Les diagrammes correspondent aux classes persistées et aux services réellement utilisés.

Exigence	Réponse apportée
Sécurité	CSRF, sessions, hashage Django, permissions et validation serveur
Intégrité	Contraintes SQL, unicité, FK, PROTECT et transactions atomic
Ergonomie	Responsive Bootstrap, contrastes, labels, messages et confirmations
Performance	Index sur références, statuts, dates, stock et relations fréquentes
Maintenabilité	Applications séparées, services métier, templates réutilisés
Traçabilité	MouvementStock et conservation des lignes historiques
Portabilité	Python, requirements.txt, migrations et guide Windows

Règles de gestion

Les règles critiques sont centralisées afin d'éviter des comportements différents selon les pages.

Références et quantités

La référence interne est unique, normalisée en majuscules et contrôlée par une expression régulière. Les prix et quantités ne peuvent être négatifs. La dimension complète doit correspondre aux valeurs largeur, hauteur, diamètre, charge et vitesse.

Commande

Une commande créée reste EN_ATTENTE. La confirmation est la seule transition qui retire le stock. Les états LIVREE et ANNULEE sont finaux. Une commande confirmée peut être annulée seulement avant préparation ; le retour de stock ne peut être réalisé qu'une fois.

Approvisionnement

Un approvisionnement BROUILLON ou COMMANDE peut être modifié ou annulé. Le passage à RECU augmente le stock, crée un mouvement d'entrée pour chaque ligne et enregistre la date de réception. Une seconde réception est refusée.

- Aucun client ne consulte la commande d'un autre client.
- Aucune quantité n'est modifiée sans mouvement traçable.
- Les suppressions historiques sont remplacées par archivage ou annulation.

Critères d'acceptation et scénarios

Chaque objectif important est associé à un résultat observable.

Scénario client

Le client se connecte, filtre le catalogue, ouvre une fiche, ajoute une quantité disponible, consulte le panier, choisit le mode de réception et de paiement, puis enregistre la commande. Il retrouve ensuite uniquement ses propres commandes.

Scénario employé

L'employé consulte les commandes en attente, confirme une commande, observe la diminution du stock et les mouvements de sortie, poursuit la préparation puis la livraison. Il réceptionne aussi les approvisionnements et surveille les alertes.

Scénario administrateur

L'administrateur dispose des opérations de l'employé et gère les comptes, marques, catégories et fournisseurs. Les actions d'archivage conservent les relations historiques.

- Stock insuffisant : opération refusée avec message.
- Panier vide : validation impossible.
- Double confirmation ou réception : exception métier.
- Accès non autorisé : redirection ou réponse 404 selon le contexte.

03

Analyse et conception

Ce chapitre présente l'architecture Django MVT, les diagrammes UML, le modèle de données et les contraintes qui relient la conception au code final. Les figures sont produites en vectoriel pour rester lisibles dans le PDF.

Architecture Django MVT

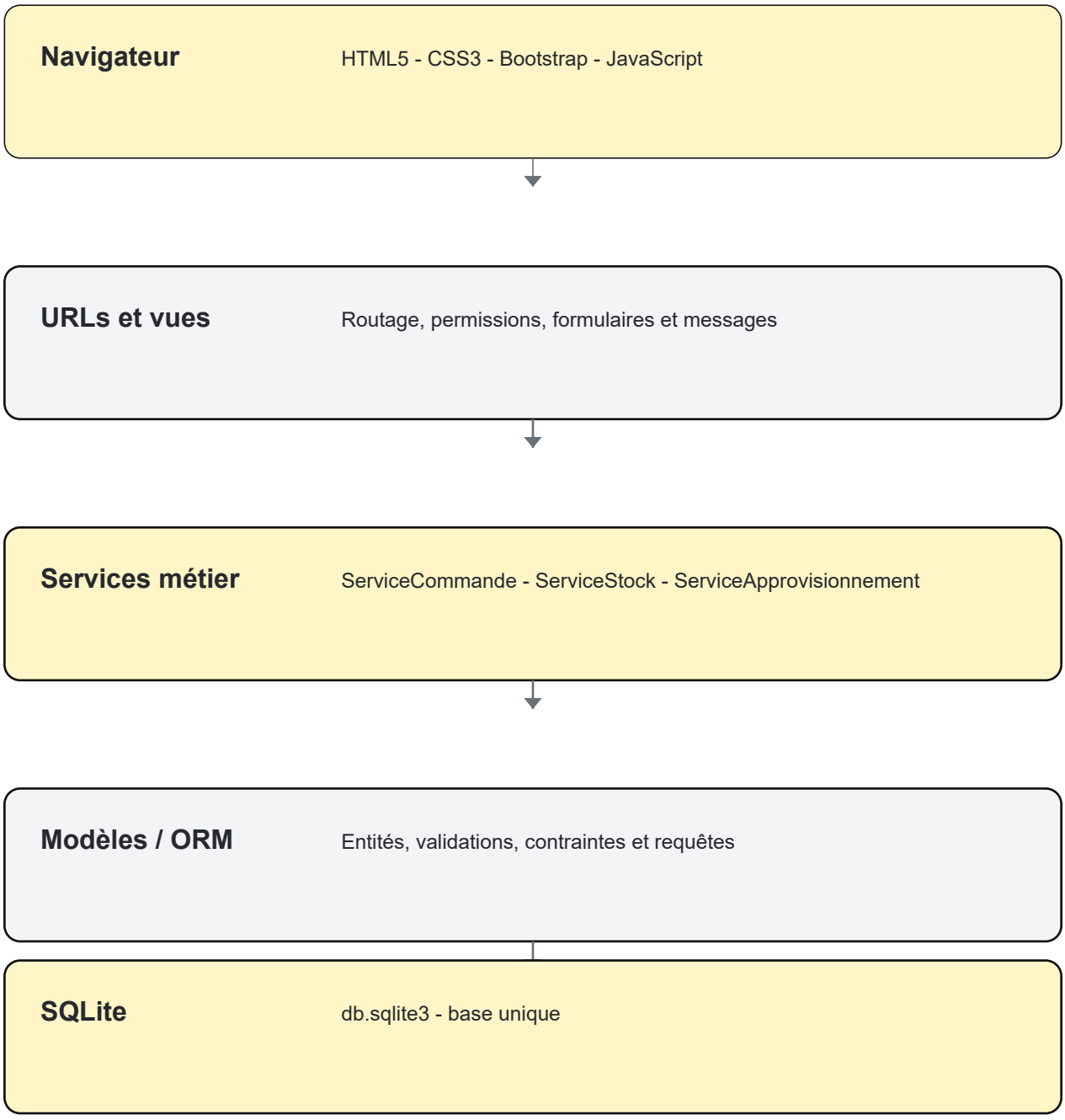


Figure 1 - Les services isolent la logique critique des vues et des templates.

Diagramme de cas d'utilisation

Figure vectorielle - qualité d'impression et zoom sans perte

Diagramme de cas d'utilisation global

Acteurs, périmètre fonctionnel et dépendances UML <<include>> / <<extend>>

MANUGOM Stock - UML

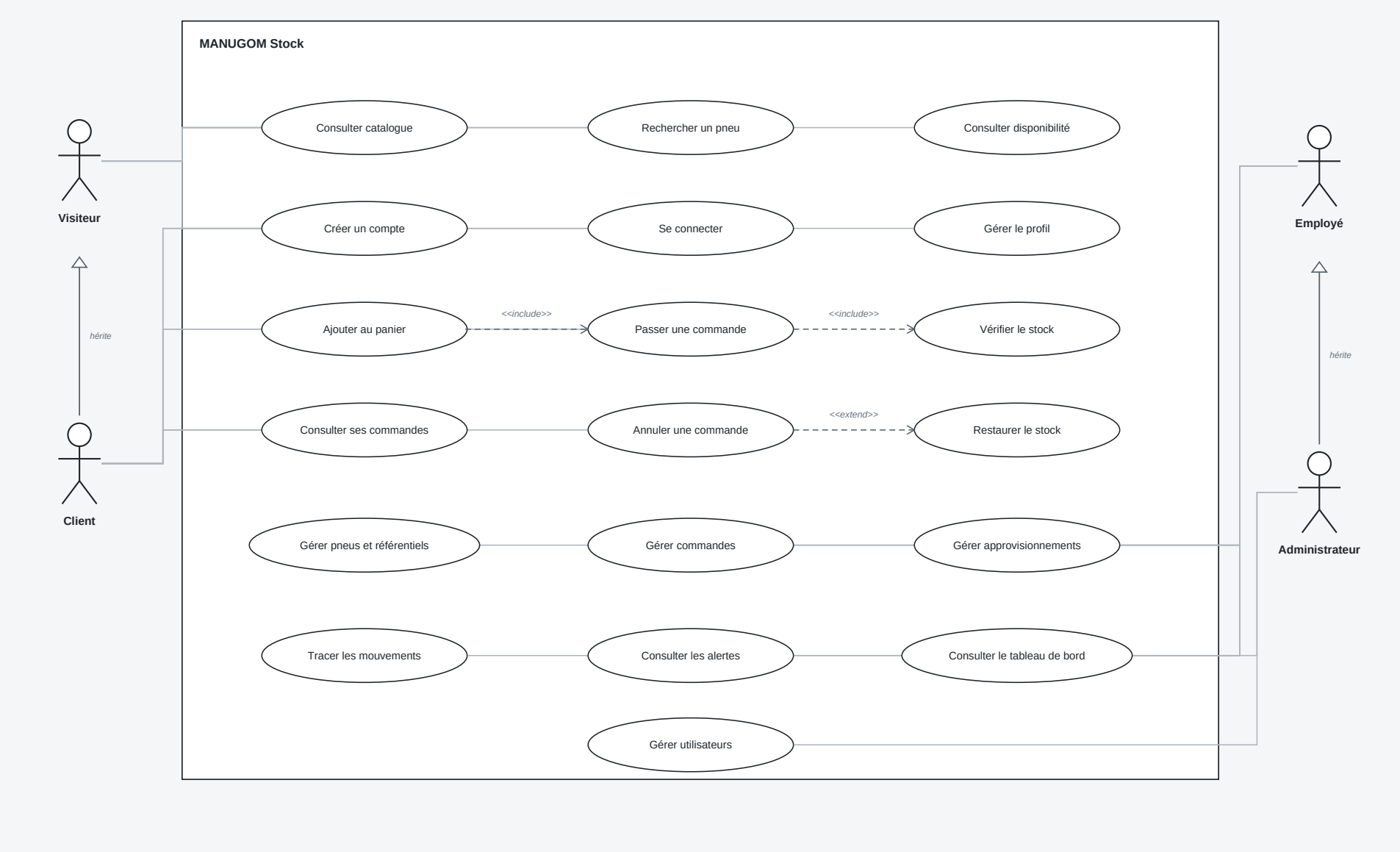


Figure 2 - Cas d'utilisation global et héritage des acteurs.

Les relations include et extend sont limitées aux dépendances réellement justifiées.

Diagramme de classes

Figure vectorielle - qualité d'impression et zoom sans perte

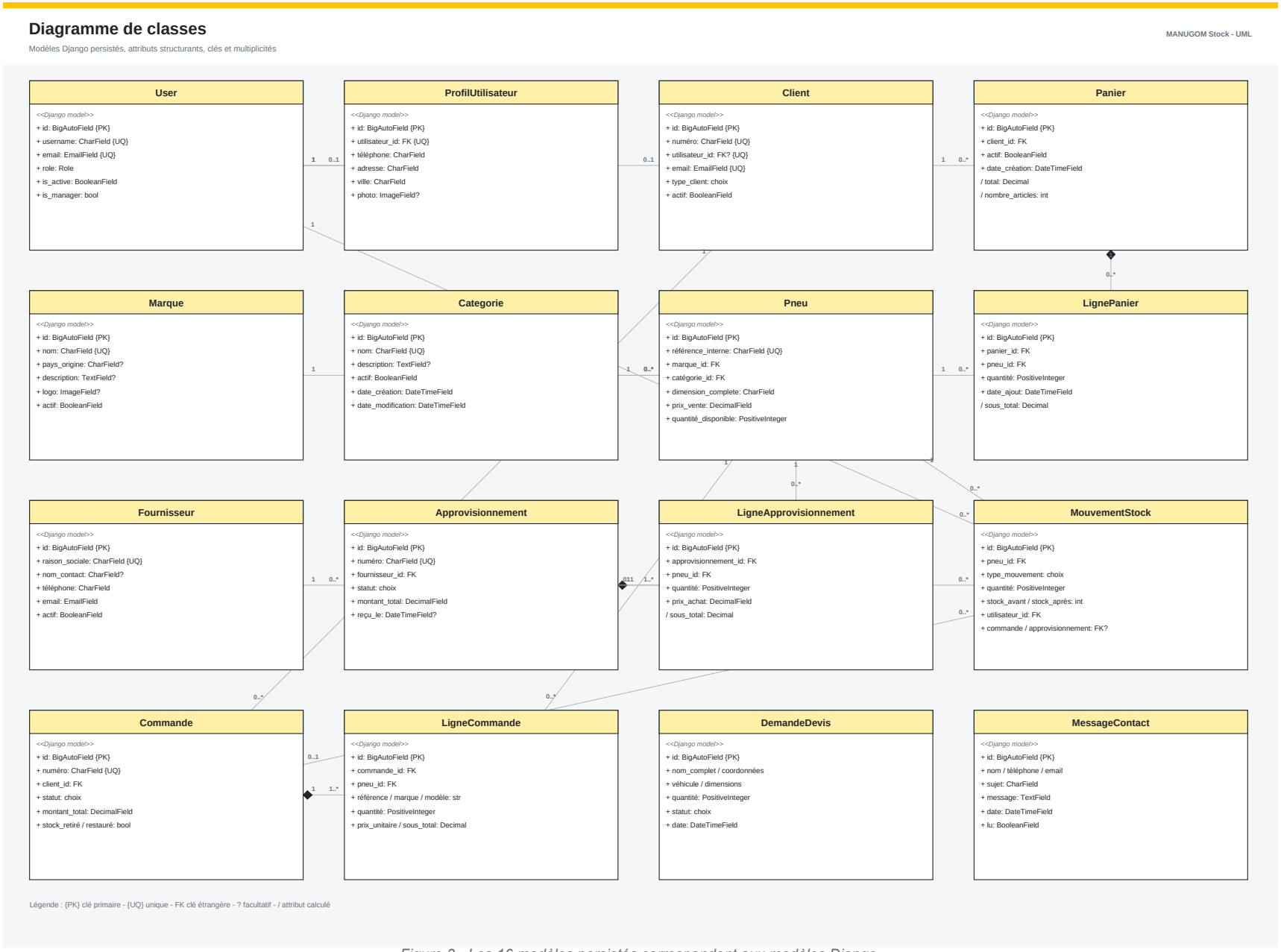


Figure 3 - Les 16 modèles persistés correspondent aux modèles Django.
PK, unicité, clés étrangères, attributs calculés et multiplicités sont explicités.

Diagramme de séquence

Figure vectorielle - qualité d'impression et zoom sans perte

Diagramme de séquence - passage et confirmation d'une commande

MANUGOM Stock - UML

Interactions HTTP, services métier, transaction Django ORM et persistance SQLite

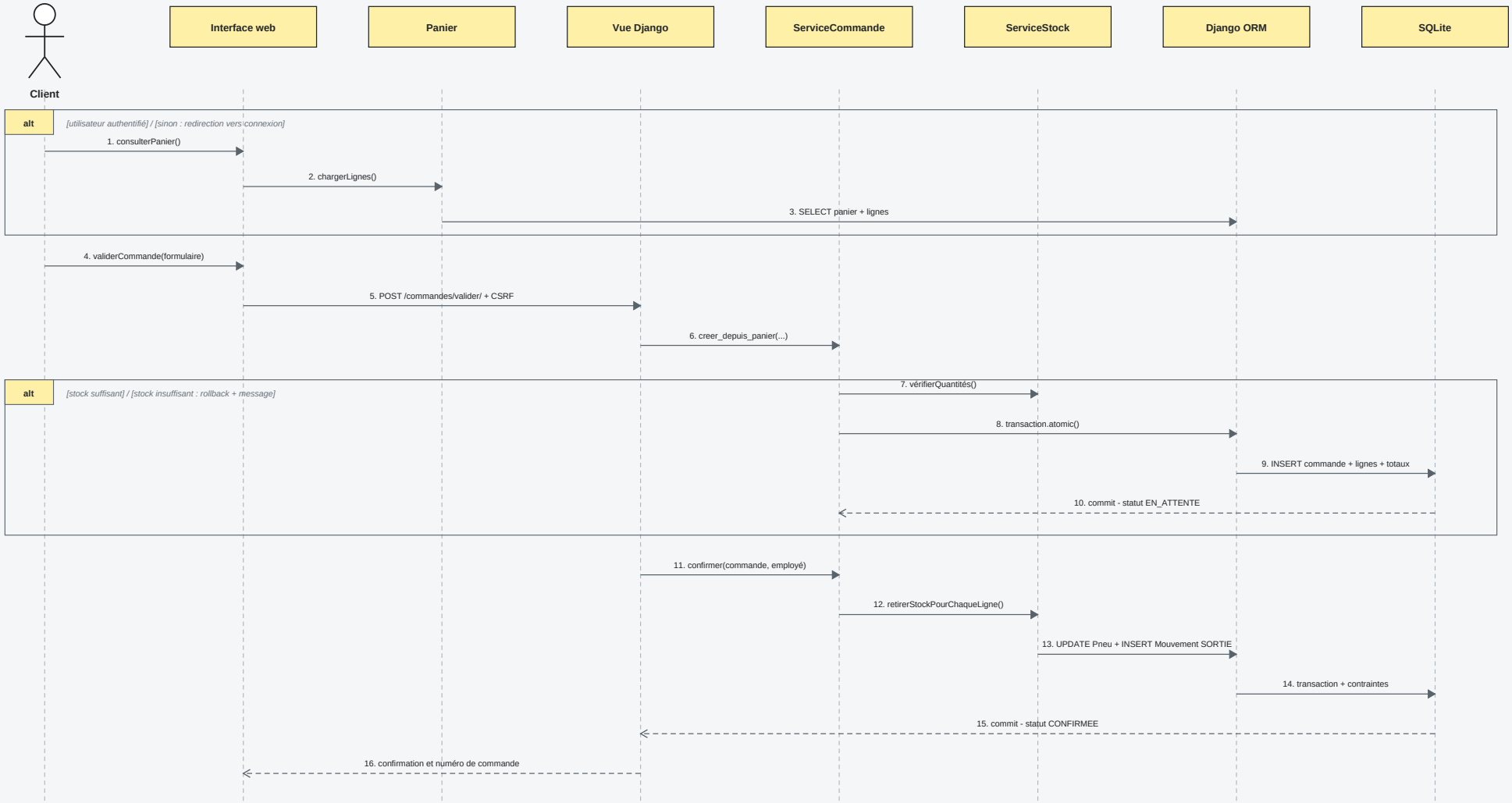


Figure 4 - Création EN_ATTENTE puis confirmation transactionnelle.
Les fragments alternatifs couvrent l'authentification et le stock insuffisant.

Diagramme d'activité

Figure vectorielle - qualité d'impression et zoom sans perte

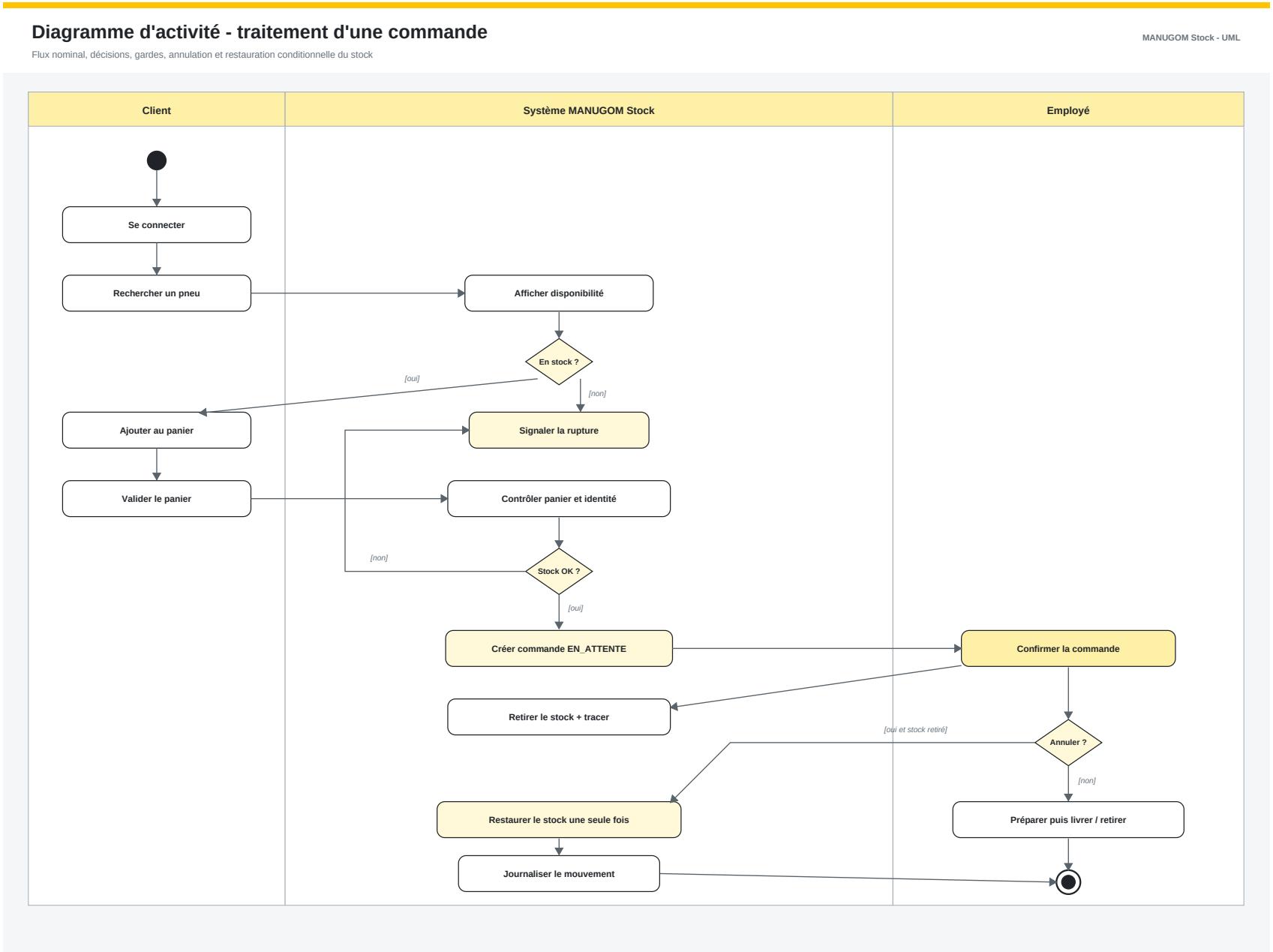


Figure 5 - Flux de commande, décisions et restauration conditionnelle.
Les gardes montrent les branches de rupture, d'annulation et de livraison.

Diagramme de déploiement

Figure vectorielle - qualité d'impression et zoom sans perte

Diagramme de déploiement

Topologie réelle de l'application Django et persistance locale SQLite

MANUGOM Stock - UML

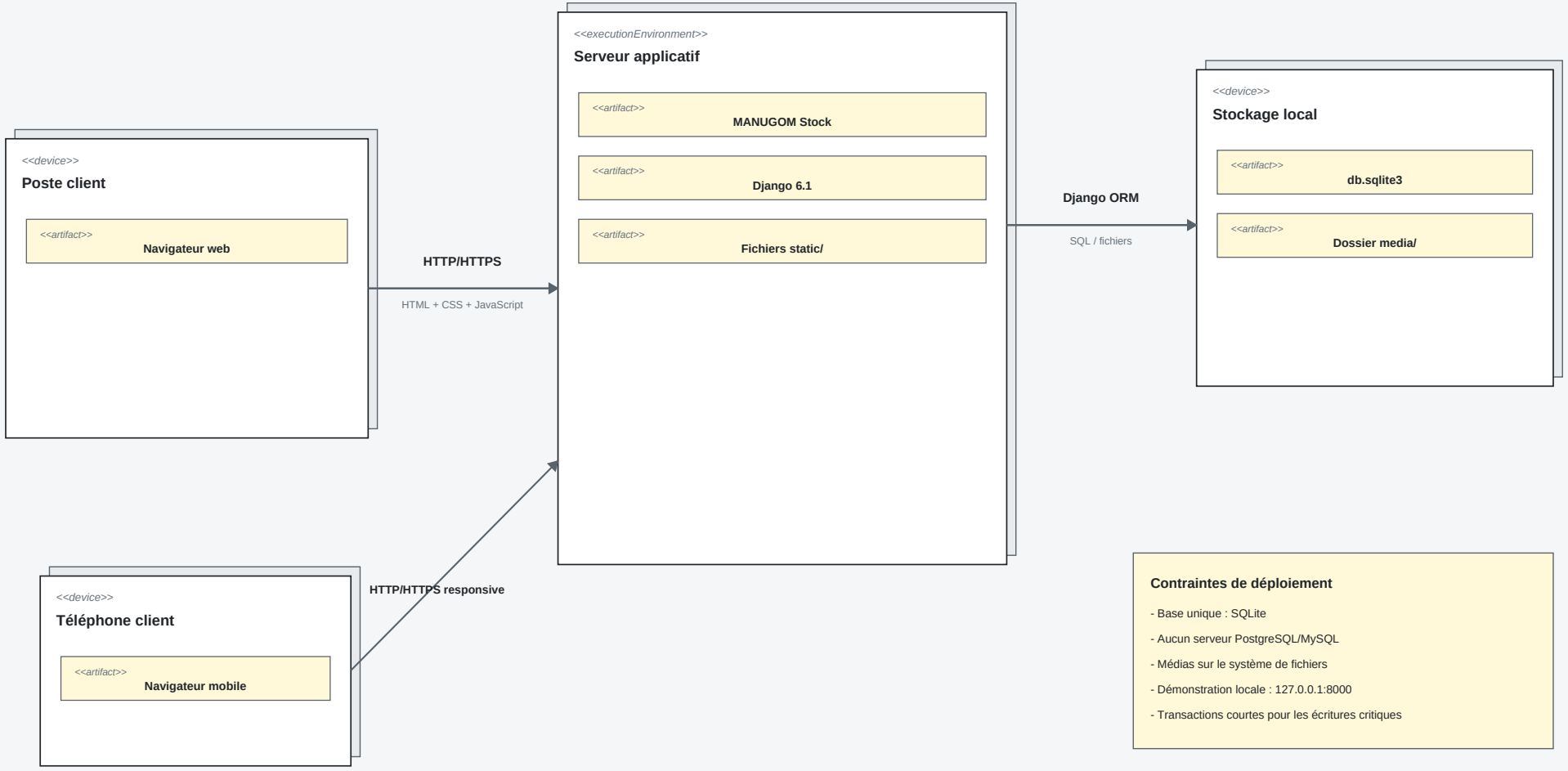


Figure 6 - Navigateur, serveur Django et stockage SQLite local.

Aucun serveur PostgreSQL ou MySQL n'est représenté.

MCD - vue relationnelle synthétique

Figure vectorielle - qualité d'impression et zoom sans perte

MCD / vue relationnelle synthétique

Entités principales et cardinalités cohérentes avec les clés étrangères Django

MANUGOM Stock - UML

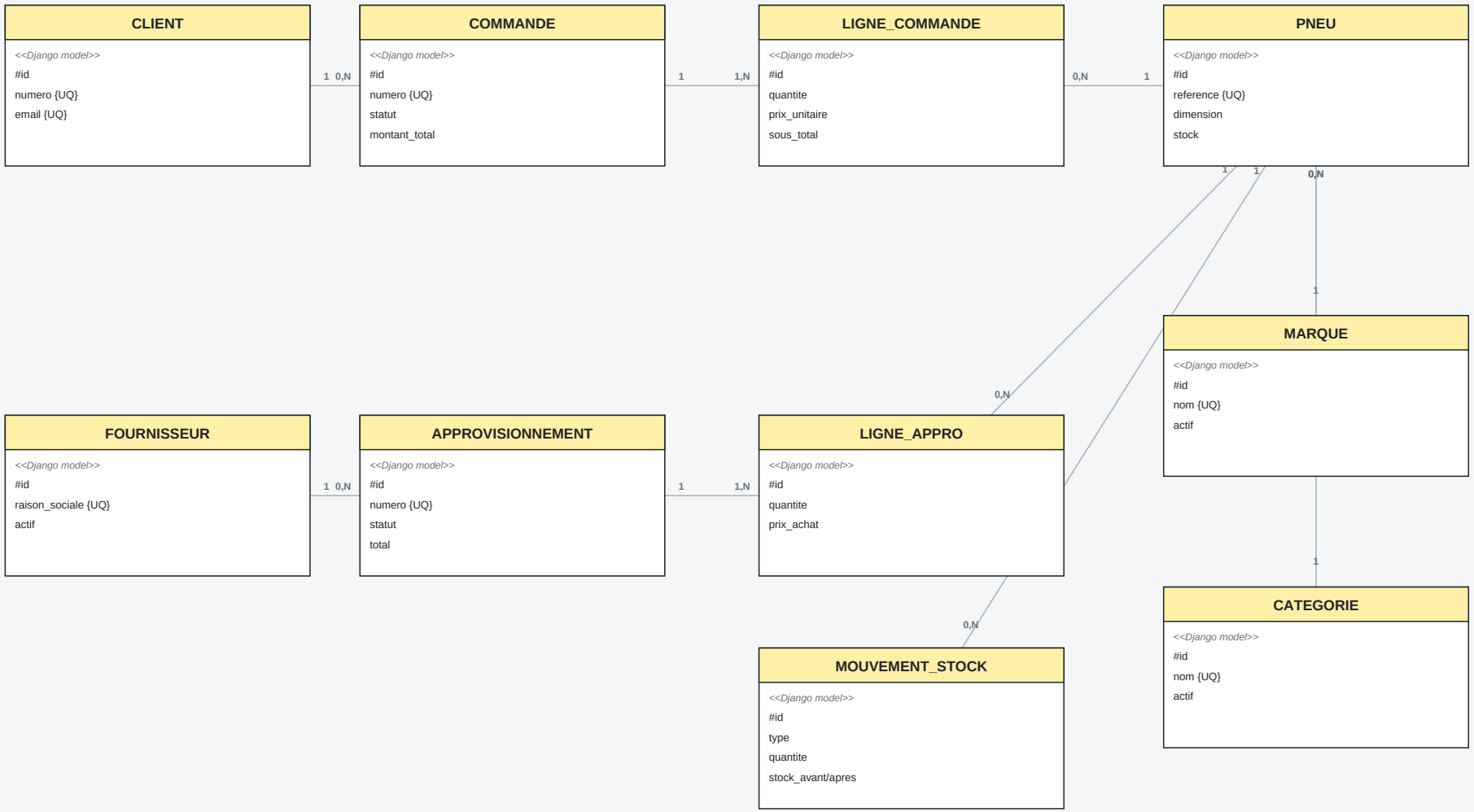


Figure 7 - Entités métier majeures et cardinalités.

Le MCD est cohérent avec les FK et contraintes des modèles Django.

Modèle logique de données

Le MLD traduit les associations en clés étrangères et contraintes relationnelles.

Principe

Django crée une clé primaire BigAutoField pour les modèles métier. Les relations PROTECT empêchent la disparition d'un pneu, d'une commande ou d'un utilisateur lorsqu'un historique de stock dépend de cet élément. Les relations CASCADE sont réservées aux composants dont la durée de vie dépend du parent, comme les lignes d'un panier.

Historisation

LigneCommande conserve des chaînes de caractères et des montants indépendants de l'état courant du pneu. Une modification ultérieure du prix ou du modèle ne réécrit donc pas une commande passée.

Table	Clé primaire	Clés étrangères principales	Contraintes
inventory_pneu	id	marque_id, categorie_id	reference_interne UQ, stock >= 0
customers_client	id	utilisateur_id	numero UQ, email UQ
orders_panier	id	client_id	un seul panier actif/client
orders_lignepanier	id	panier_id, pneu_id	paire panier/pneu UQ
orders_commande	id	client_id	numero UQ, statuts contrôlés
orders_lignecommande	id	commande_id, pneu_id	instantané prix/référence
inventory_mouvementstock	id	pneu, commande?, appro?, utilisateur	quantité >= 1
suppliers_approvisionnement	id	fournisseur, responsable	numero UQ
suppliers_ligneapprovisionnement	id	approvisionnement, pneu	quantité >= 1

Contraintes d'intégrité et dictionnaire

Les validations sont appliquées avant l'enregistrement et complétées par des contraintes SQLite.

Intégrité de domaine

Les choix Django bornent les statuts, saisons, rôles et types de mouvements. Des validateurs contrôlent les images, dimensions et références. Les champs numériques positifs empêchent des valeurs incohérentes.

Intégrité référentielle

Les relations utilisent CASCADE, SET_NULL ou PROTECT selon la valeur historique. L'archivage est préféré à la suppression lorsqu'une référence, un client ou un fournisseur a déjà participé à une opération.

Table.champ	Type	Obligatoire	Règle
Pneu.reference_interne	varchar(60)	Oui	Unique, majuscules, sans espace
Pneu.dimension_complete	varchar(30)	Oui	Format NNN/NN RNN NN[L]
Pneu.prix_vente	decimal(10,2)	Oui	>= prix_achat et >= 0
Pneu.quantite_disponible	integer	Oui	>= 0
Commande.numero	varchar(25)	Oui	Unique, CMD-AAAA-NNNNN
Commande.statut	varchar(25)	Oui	Transition autorisée seulement
LigneCommande.prix_unitaire	decimal(10,2)	Oui	Prix au moment de la commande
MouvementStock.stock_avant	integer	Oui	Instantané avant opération
Approvisionnement.recu_le	datetime	Non	Défini lors de la réception

04

Réalisation et développement

Ce chapitre décrit l'environnement, l'organisation des applications, les modules fonctionnels, la sécurité, les interfaces et les décisions prises pendant l'implémentation de MANUGOM Stock.

Environnement et technologies

La stack respecte la contrainte Django + SQLite et évite une seconde base de données.

Environnement

Le projet est organisé pour Windows avec un environnement virtuel .venv, un fichier requirements.txt, des migrations et une commande seed_demo. Le serveur de développement écoute localement sur 127.0.0.1:8000.

Choix

Django fournit une structure complète pour l'authentification, les formulaires, l'ORM et les tests. SQLite simplifie la démonstration et le transport du projet. Bootstrap permet une adaptation rapide aux écrans, tandis que Chart.js est utilisé uniquement pour la visualisation côté navigateur.

Technologie	Rôle dans le projet
Python	Langage du backend et des scripts de gestion
Django 6.1.1	Framework MVT, ORM, formulaires, authentification et tests
SQLite	Base de données unique db.sqlite3
HTML5 / CSS3	Structure sémantique et identité visuelle
JavaScript	Interactions légères et confirmations
Bootstrap 5	Grille responsive et composants d'interface
Bootstrap Icons	Pictogrammes fonctionnels
Chart.js	Cinq graphiques alimentés par SQLite
StarUML / SVG	Modèle .mdj et diagrammes vectoriels à valider dans StarUML

Structure du projet Django

Huit applications séparent les domaines fonctionnels.

Applications

accounts gère le User personnalisé et les rôles ; inventory contient marques, catégories, pneus, mouvements et ServiceStock ; customers stocke les fiches clients ; suppliers gère les fournisseurs et approvisionnements ; orders contient paniers, commandes et ServiceCommande ; dashboard regroupe l'espace gestion ; quotes gère devis et contact ; core fournit les pages institutionnelles et seed_demo.

Templates et fichiers statiques

Les templates héritent d'une base commune et réutilisent une carte pneu. Les fichiers CSS et JavaScript sont centralisés dans static/. Les images de démonstration sont distinctes des médias téléversés, stockés sous media/.

Documentation

Le dossier docs contient le rapport source, les spécifications UML, le MCD, le MLD, le dictionnaire et les résultats de tests. Le dossier output/pdf contient le présent rapport final.

- La logique critique n'est pas concentrée dans les vues.
- Les URL sont regroupées par espace de noms.
- Les migrations reflètent les modèles persistés.

Authentification, rôles et sécurité

Django assure le hashage des mots de passe, les sessions et la protection CSRF.

Utilisateur personnalisé

Le modèle User étend AbstractUser et ajoute le rôle ADMIN, EMPLOYEE ou CLIENT. Un superutilisateur est automatiquement considéré comme administrateur. La propriété is_manager simplifie le contrôle des espaces de gestion.

Permissions

Les décorateurs manager_required et admin_required protègent les vues. Les requêtes de détail d'une commande filtrent par client lorsqu'un client est connecté. L'accès à une commande étrangère retourne 404 pour ne pas divulguer son existence.

Sécurité des formulaires

Les formulaires POST incluent un jeton CSRF, les valeurs sont validées côté serveur et les quantités sont recalculées depuis SQLite. Les cookies de session et CSRF utilisent l'option HttpOnly lorsque cela s'applique, le sniffing de contenu est désactivé et les pages ne peuvent pas être intégrées dans une iframe.

Gestion des pneus et catalogue

Le module inventory associe validation métier, recherches et interface publique.

CRUD

Le gestionnaire ajoute et modifie les pneus avec un ModelForm. L'archivage passe le statut à ARCHIVE au lieu de supprimer l'historique. Les marques et catégories suivent le même principe actif/inactif.

Catalogue

Le visiteur filtre par référence, mot-clé, marque, catégorie, largeur, hauteur, diamètre, saison, prix et disponibilité. Les tris couvrent le prix, la marque, la date et le stock. Les résultats sont paginés par groupes de neuf.

Validation

La méthode clean vérifie la cohérence de la dimension et des prix. Le save normalise la référence et l'indice de vitesse. Des index accélèrent les filtres sur dimension, marque, catégorie, saison, statut et quantité.

Panier et commandes clients

Le panier actif est conservé en base et associé à une fiche client.

Panier

L'ajout refuse une quantité nulle, négative ou supérieure au stock. Une contrainte unique interdit de dupliquer un pneu dans un même panier. Le total est calculé à partir des prix en base, jamais à partir d'une valeur fournie par JavaScript.

Création de commande

ServiceCommande ouvre une transaction, contrôle le panier, crée Commande et LigneCommande, puis recalcule le total. Le panier devient inactif et une nouvelle commande reste EN_ATTENTE sans retirer immédiatement le stock.

Transitions

La confirmation, la préparation, la disponibilité, l'expédition, la livraison et l'annulation suivent une table explicite. Les lignes conservent leur prix historique. Le correctif du parcours avant connexion garantit un retour vers une page GET valide au lieu d'une réponse HTTP 405.

Stock et approvisionnements

Les services métier constituent la frontière unique des mises à jour de quantité.

ServiceStock

La méthode appliquer_mouvement verrouille la ligne de pneu dans la transaction, calcule le nouveau stock, refuse une valeur négative, met à jour la quantité puis crée MouvementStock avec les instantanés avant et après.

Confirmation et retour

La confirmation relit les pneus et retire chaque quantité. Une sortie est associée à la commande. Si une commande confirmée est annulée avant préparation, chaque ligne produit un retour. Les drapeaux stock_retire et stock_restaure empêchent la répétition.

Réception fournisseur

ServiceApprovisionnement vérifie le statut, applique les mouvements d'entrée, fixe RECU et enregistre recu_le. Les lignes conservent le prix d'achat et permettent de recalculer le montant total.

Tableau de bord, devis et contact

Les indicateurs donnent une vue synthétique à partir des données réellement présentes dans SQLite.

Indicateurs

Le tableau de bord affiche le nombre de références, la quantité et la valeur du stock, les clients actifs, les commandes du jour, les commandes en attente ou livrées, le chiffre d'affaires livré estimé, les stocks faibles et les ruptures.

Graphiques

Chart.js représente les ventes mensuelles, les stocks par marque, les commandes par statut, les mouvements agrégés et les produits les plus vendus. Les séries sont sérialisées dans le template par json_script.

Relation client

Les demandes de devis et messages de contact sont enregistrés en base et consultables dans l'espace de gestion. Le téléphone utilise un lien tel: et WhatsApp un lien wa.me. Aucune API externe n'est exécutée par l'application.

Identité visuelle et interface

Choix graphiques appliqués au site responsive





#FFC400 #111111 Anthracite Blanc

Principes appliqués

- Contraste fort jaune/noir, cohérent avec l'univers automobile et les images de référence.
- Composants Bootstrap responsives : navigation, cartes produit, tableaux et formulaires.
- Badges de stock vert, jaune et rouge ; messages explicites et actions reliées à des vues Django.
- Visuels originaux de démonstration sans logo commercial copié ni référence officielle inventée.

05

Tests, résultats et validation

La validation combine tests unitaires, tests de services, tests de permissions et tests de rendu. Les résultats présentés sont issus de commandes réellement exécutées sur la version documentée.

Stratégie de tests

La suite de tests utilise la base SQLite temporaire créée par Django.

Modèles et validation

Les tests contrôlent la création, la modification et l'archivage d'un pneu, l'unicité de la référence, le format de dimension, la relation entre prix d'achat et prix de vente et l'impossibilité d'obtenir un stock négatif.

Services transactionnels

Les scénarios vérifient la création d'une commande, les totaux, la conservation du prix, la confirmation, la sortie, la double confirmation, l'annulation, le retour unique, la réception fournisseur, l'augmentation du stock et la double réception.

Vues et permissions

Les rôles, l'inscription, l'isolation des commandes, les pages publiques, client et gestion sont vérifiés. Des tests spécifiques confirment le retrait de la carte et des exports ainsi que le parcours complet allant du panier à la commande.

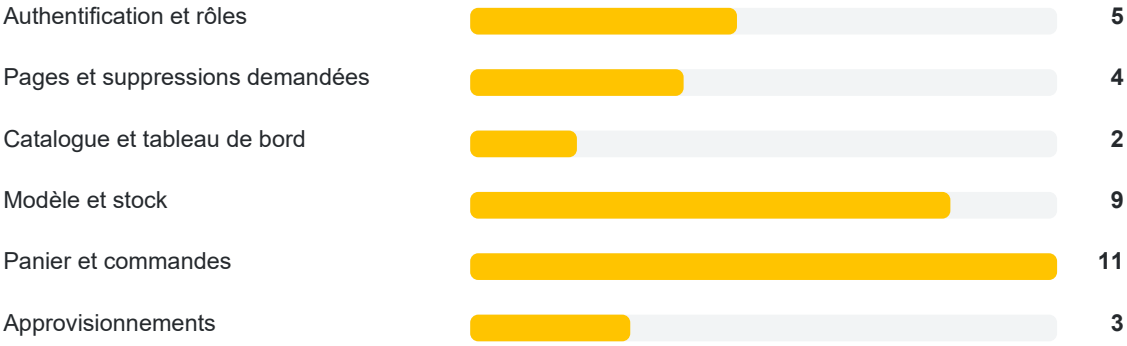
- Commande : python manage.py test --verbosity 2
- Contrôle : python manage.py check
- Migrations : python manage.py makemigrations --check --dry-run

Résultats des tests et données de démonstration

34 / 34 **tests réussis**

0 échec - 0 erreur - System check sans anomalie

Couverture de la suite



Jeu de démonstration présent dans SQLite

Entité	Nombre	Entité	Nombre
Utilisateurs	13	Clients	10
Marques	8	Catégories	5
Pneus	20	Fournisseurs	4
Commandes	9	Approvisionnements	4
Mouvements	39		

La commande de test finale a créé une commande à partir du panier via les vues HTTP, puis a vérifié le total et la ligne enregistrée. Le contrôle SQLite PRAGMA integrity_check retourne ok.

Problèmes rencontrés et corrections

Les anomalies observées ont été traitées avant la production de ce rapport.

Concurrence SQLite

SQLite verrouille plus largement les écritures qu'une base serveur. La réponse consiste à garder les transactions courtes, à revérifier le stock au dernier moment et à documenter la limite pour une évolution future.

Historique et suppression

Une suppression physique pouvait casser les commandes ou mouvements. Les relations PROTECT, les instantanés de ligne et l'archivage ont été retenus pour préserver les preuves métier.

Redirection après connexion

Un ajout au panier avant authentification redirigeait vers l'URL POST d'ajout, ensuite appelée en GET par le navigateur, ce qui produisait HTTP 405. Le bouton propose désormais une connexion vers la fiche du pneu et la vue protège également les requêtes directes.

Évolutions demandées

La zone réservée à une carte et toutes les fonctions de téléchargement de données ont été supprimées. Les vues, URL, templates, tests, UML et documents ont été remis en cohérence.

Validation finale et limites

Les contrôles finaux confirment la cohérence de la version livrée.

Contrôles réussis

Les 34 tests passent, le système Django ne signale aucune anomalie, aucune migration supplémentaire n'est détectée et le contrôle d'intégrité SQLite retourne ok. Les pages principales ont répondu en HTTP 200 pendant les vérifications.

UML

Les diagrammes de cas d'utilisation, classes, séquence, activité et déploiement sont fournis en SVG vectoriel. Le fichier manugom-stock.mdj possède des identifiants uniques et des références résolues. StarUML n'étant pas disponible dans l'environnement, une ouverture finale dans StarUML reste nécessaire avant la remise.

Limites

Le serveur Django de développement n'est pas un serveur de production. Les emails utilisent la console. SQLite n'est pas destiné à une forte concurrence. Aucun paiement, transporteur, API WhatsApp ou carte externe n'est intégré.

Guide utilisateur synthétique

Le parcours dépend du rôle connecté.

Client

Ouvrir le catalogue, utiliser les filtres, consulter une fiche puis se connecter si nécessaire. Ajouter une quantité, ouvrir le panier, modifier ou retirer les lignes, valider l'adresse, la ville, la réception et le paiement, puis consulter Mes commandes.

Employé

Se connecter depuis la page normale. Ouvrir Gestion, consulter les alertes, créer ou modifier les références, enregistrer les mouvements autorisés, traiter les commandes et réceptionner les approvisionnements.

Administrateur

Utiliser l'espace Gestion pour les opérations métier et /django-admin/ pour une vue technique complète. Gérer les employés et utilisateurs avec prudence. Préférer l'archivage à la suppression d'un élément déjà utilisé.

- Toujours vérifier la référence et la dimension avant une opération.
- Ne jamais partager les mots de passe de démonstration pour un usage réel.
- Consulter les mouvements pour expliquer toute variation de stock.

Conclusion générale et perspectives

MANUGOM Stock répond au besoin académique par une application complète, persistante et testée.

Bilan

Le projet couvre le catalogue, les référentiels, le panier, les commandes, les clients, les fournisseurs, les approvisionnements, les mouvements, les alertes, les devis, le contact et les statistiques. Les services métier protègent les opérations critiques et les rôles limitent les accès.

Apports

La réalisation démontre la mise en œuvre d'une architecture Django MVT, d'une modélisation relationnelle, de transactions, de validations, d'une interface responsive et d'une démarche de test. La documentation relie les besoins, la conception et le code final.

Perspectives

Une évolution vers PostgreSQL peut être envisagée en cas de forte concurrence. D'autres extensions possibles sont un serveur WSGI/ASGI de production, des notifications email autorisées, un système de codes-barres, un inventaire cyclique, un suivi transporteur et des tests de navigateur automatisés.

Bibliographie et webographie

Les sources techniques privilégiées sont les documentations officielles. Consultation : septembre 2026.

Méthode de citation

Les documents internes fournis ont servi de référence de structure et de mise en forme. Leur contenu métier n'a pas été repris. Les documentations en ligne ont été utilisées pour confirmer le rôle des technologies et les pratiques générales décrites dans le rapport.

Réf.	Source
[1]	Django Software Foundation, Documentation Django - https://docs.djangoproject.com/en/6.0/topics/
[2]	Django Software Foundation, Authentification - https://docs.djangoproject.com/en/6.0/topics/auth/
[3]	SQLite Consortium, Documentation SQLite - https://www.sqlite.org/docs.html
[4]	Bootstrap Team, Documentation Bootstrap 5.3 - https://getbootstrap.com/docs/5.3/
[5]	Chart.js Contributors, Getting Started - https://www.chartjs.org/docs/latest/getting-started/
[6]	StarUML, Basic Concepts et CLI - https://docs.staruml.io/
[7]	EMSI, Guide de rédaction du rapport PFA, document pédagogique fourni.
[8]	EMSI, Structure et recommandations de mise en forme du rapport PFA, document fourni.
[9]	Rapport PFA SecureBridge, document de référence fourni pour la structure et la couverture.

Annexe A - Installation, lancement et comptes

Commandes Windows PowerShell

```
cd "C:\chemin\vers\manugom-stock"
py -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt
python manage.py migrate
python manage.py seed_demo
python manage.py test
python manage.py runserver
```

Comptes de démonstration

Rôle	Identifiant	Mot de passe	Accès
Administrateur	admin	Admin@2026!	Gestion + Django admin
Employé	employe1 / employe2	Employe@2026!	Espace Gestion
Client	client1 à client10	Client@2026!	Catalogue, panier, commandes

URL locale : http://127.0.0.1:8000/ - Administration Django : /django-admin/. Ces identifiants sont exclusivement destinés à la démonstration et doivent être remplacés avant tout déploiement réel.

Annexe B - Modèles et transitions

Le projet contient seize modèles persistés et trois services métier principaux.

Modèles

User, ProfilUtilisateur, Client, Marque, Categorie, Pneu, Panier, LignePanier, Commande, LigneCommande, Fournisseur, Approvisionnement, LigneApprovisionnement, MouvementStock, DemandeDevis et MessageContact.

Services

ServiceStock applique les mouvements ; ServiceCommande crée, confirme, fait évoluer et annule les commandes ; ServiceApprovisionnement réceptionne les achats fournisseurs.

Transitions de commande

EN_ATTENTE vers CONFIRMEE ou ANNULEE ; CONFIRMEE vers EN_PREPARATION ou ANNULEE ; EN_PREPARATION vers PRETE ; PRETE vers EXPEDIEE ou LIVREE pour un retrait ; EXPEDIEE vers LIVREE. LIVREE et ANNULEE sont finales.

- Confirmation : deuxième contrôle, retrait et mouvements SORTIE.
- Annulation confirmée : retours uniques si la préparation n'a pas commencé.
- Réception fournisseur : entrées uniques et statut RECU.